



Introduction

AdEase is an AI-driven advertising and marketing infrastructure that helps businesses maximize clicks at minimal cost. With a 3-step digital advertising solution powered by AI modules — Design, Dispense, and Decipher — AdEase enables brands to promote themselves easily, effectively, and economically.

As part of the Data Science team at AdEase, the goal is to leverage historical web traffic data from Wikipedia to forecast future page views. This predictive capability enables smarter ad placement strategies, customized for different regions and languages — helping clients gain better visibility and engagement across relevant content.

To achieve this, you are provided with daily view data for over 145,000 Wikipedia pages spanning 550 days, and tasked with building time series models to forecast future views. These insights will directly support the optimization of digital ad campaigns by identifying high-traffic opportunities, segmented by language, region, device type, and access origin.

Data Dictionary

You are provided with two datasets:

1. train_1.csv

Each row represents a unique Wikipedia page.

Each column (after the first) represents a specific date.

The values are the number of views on that date.

The page name format follows this structure:

`<PAGE_TITLE>.wikipedia.org<ACCESS_TYPE>_<ACCESS_ORIGIN>`

From this, we can extract:

Page Title (e.g., Barack_Obama)

Language (e.g., en for English, de for German)

Access Type (e.g., desktop, mobile)

Access Origin (e.g., spider, user/browser)

2. Exog_Campaign_eng.csv

Contains a binary indicator (1 or 0) for each date.

1 indicates the presence of a campaign or significant event that could have influenced page views.

Applicable only for English language pages.

Used as an exogenous variable in time series models like SARIMAX and Prophet.

🧩 Problem Statement

AdEase aims to predict future Wikipedia page views across multiple languages and regions in order to optimize ad placements for its clients.

You are tasked with:

Analyzing daily page view trends across 145k+ Wikipedia pages.

Forecasting traffic using time series models (ARIMA, SARIMAX, Prophet).

Segmenting insights by language, device, and access origin to inform region-specific ad strategies.

Using event-level data (e.g., campaign days) to improve forecast accuracy for English pages.

Ultimately, this project will provide the foundation for data-driven, cost-effective ad targeting, allowing clients to invest in content areas with the highest potential visibility.

Data : <https://www.scaler.com/academy/mentee-dashboard/discussion-forum/p/ask-me-anything-business-case-adease/21147>

```
In [184... # Library
# libraries to analyze data
import numpy as np
import pandas as pd

# libraries to visualize data
import matplotlib.pyplot as plt
import seaborn as sns

import re

import statsmodels.api as sm
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf
```

```

from sklearn.metrics import (
    mean_squared_error as mse,
    mean_absolute_error as mae,
    mean_absolute_percentage_error as mape
)

from statsmodels.tsa.arima.model import ARIMA

```

In [185... *#Loading the Data*

```
df = pd.read_csv('train_1.csv')
```

In [186... **def** explore_dataframe(df, df_name='DataFrame'):

```

    print('*' * 60)
    print(f' Info for {df_name}')
```

print('*' * 60)

```

    print(df.info())
    print('\n' + '*' * 60)
    print(f' Shape of {df_name}: {df.shape}')
```

print('*' * 60)

```

    print(f' Null values in {df_name}: \n{df.isna().sum()}')
```

print('*' * 60)

```

    print(f' Unique values in each column of {df_name}: \n{df.nunique()}')
```

print('*' * 60)

```

    print(f' Duplicate rows in {df_name}: \n{df.duplicated().value_counts()}')
```

print('*' * 60 + '\n')

In [187... *# Data Analysis*

```
explore_dataframe(df, 'df')
```

```

*****
📄 Info for df
*****
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145063 entries, 0 to 145062
Columns: 551 entries, Page to 2016-12-31
dtypes: float64(550), object(1)
memory usage: 609.8+ MB
None

*****
📐 Shape of df: (145063, 551)
*****

🍷 Null values in df:
Page      0
2015-07-01    20740
2015-07-02    20816
2015-07-03    20544
2015-07-04    20654
...
2016-12-27    3701
2016-12-28    3822
2016-12-29    3826
2016-12-30    3635
2016-12-31    3465
Length: 551, dtype: int64
*****

📊 Unique values in each column of df:
Page      145063
2015-07-01    6898
2015-07-02    6823
2015-07-03    6707
2015-07-04    6995
...
2016-12-27    8938
2016-12-28    8819
2016-12-29    8761
2016-12-30    8733
2016-12-31    8826
Length: 551, dtype: int64
*****

🔄 Duplicate rows in df:
False      145063
Name: count, dtype: int64
*****

```

```

In [188... # look at the top 20 rows
df.head(5)

```

Out[188...

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04
0	2NE1_zh.wikipedia.org_all-access_spider	18.0	11.0	5.0	1.0
1	2PM_zh.wikipedia.org_all-access_spider	11.0	14.0	15.0	1.0
2	3C_zh.wikipedia.org_all-access_spider	1.0	0.0	1.0	1.0
3	4minute_zh.wikipedia.org_all-access_spider	35.0	13.0	10.0	1.0
4	52_Hz_I_Love_You_zh.wikipedia.org_all-access_s...	NaN	NaN	NaN	1.0

5 rows × 551 columns

In [189... `df.describe()`

	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05
count	1.243230e+05	1.242470e+05	1.245190e+05	1.244090e+05	1.244040e+05
mean	1.195857e+03	1.204004e+03	1.133676e+03	1.170437e+03	1.217769e+03
std	7.275352e+04	7.421515e+04	6.961022e+04	7.257351e+04	7.379612e+04
min	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	1.300000e+01	1.300000e+01	1.200000e+01	1.300000e+01	1.400000e+01
50%	1.090000e+02	1.080000e+02	1.050000e+02	1.050000e+02	1.130000e+02
75%	5.240000e+02	5.190000e+02	5.040000e+02	4.870000e+02	5.400000e+02
max	2.038124e+07	2.075219e+07	1.957397e+07	2.043964e+07	2.077211e+07

8 rows × 550 columns

In [190... `df.describe(include='object')`

	Page
count	145063
unique	145063
top	Francisco_el_matemático_(serie_de_televisión_d...
freq	1

Insight

The dataset has 8054 Wikipedia pages with daily views across 550 days.

Each row is unique — no duplicate entries.

The 'Page' column has no missing values, but every date column has some missing data, likely due to pages being unavailable or untracked on certain days.

View counts vary by day and page, showing different traffic patterns.

```
In [191... # read the file containing flag for each date indicating if those dates had a
exog_en = pd.read_csv('Exog_Campaign_eng')
```

```
In [192... explore_dataframe(exog_en, "exog_en")
```

```
*****
📄 Info for exog_en
*****
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550 entries, 0 to 549
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Exog      550 non-null      int64
dtypes: int64(1)
memory usage: 4.4 KB
None

*****
📐 Shape of exog_en: (550, 1)
*****
🍷 Null values in exog_en:
Exog      0
dtype: int64
*****
📊 Unique values in each column of exog_en:
Exog      2
dtype: int64
*****
🎨 Duplicate rows in exog_en:
True      548
False     2
Name: count, dtype: int64
*****
```

```
In [193... exog_en.head()
```

Out[193...

	Exog
0	0
1	0
2	0
3	0
4	0

Insight

The dataset contains 550 rows, each representing a day (aligned with the view data).

The column Exog indicates whether a campaign or significant event happened on that day (1 = campaign day, 0 = normal day).

There are no missing values.

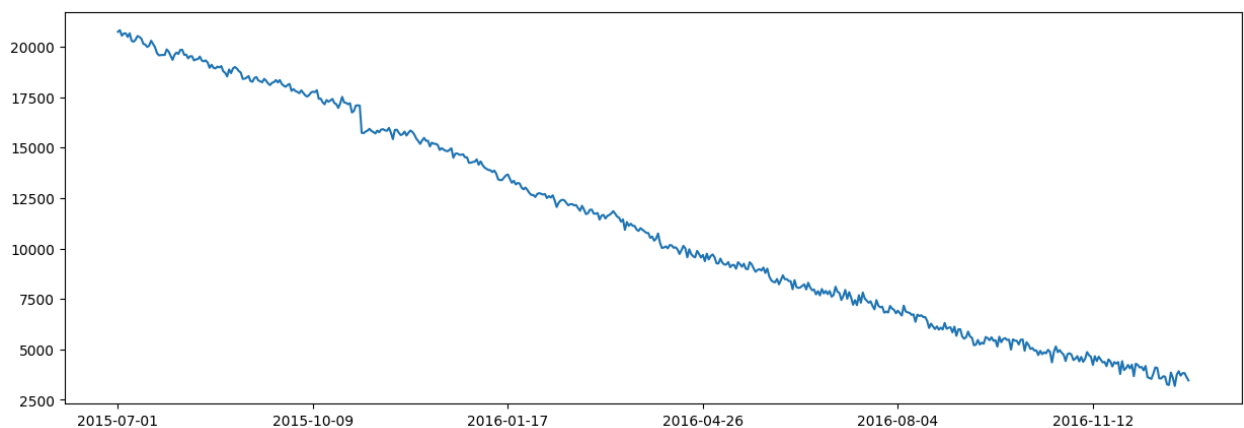
Only 2 unique values (0 and 1), as expected.

Most rows are duplicates because the value 0 (no campaign) appears frequently — only a few days had campaigns.

4. Exploratory Data Analysis

In [194...

```
date_columns = df.columns[1:]
df[date_columns].isna().sum().plot(figsize=(15,5))
plt.show()
```



Insight

The graph shows a steady decline in the number of missing values per day over

time.

Around July 2015, there were over 1200 missing entries per day.

By end of 2016, that number drops to below 100.

This suggests:

More pages were added or tracked over time.

The dataset became more complete and consistent as time progressed.

Early days have less reliable coverage, which should be considered when modeling.

```
In [195... df[date_columns] = df.loc[:,date_columns].fillna(0)
```

```
In [196... df.isna().sum()
```

```
Out[196...      0
      Page  0
2015-07-01  0
2015-07-02  0
2015-07-03  0
2015-07-04  0
      ...  ...
2016-12-27  0
2016-12-28  0
2016-12-29  0
2016-12-30  0
2016-12-31  0
```

551 rows × 1 columns

dtype: int64

```
In [197... df.duplicated().sum()
```

```
Out[197... np.int64(0)
```

4.2. Extracting information from Page column

```
In [198... df['Page'].sample(10)
```


69281	Star_Wars:_Episode_I_-_Die_dunkle_Bedrohung_de...
32178	浮士德的微笑_zh.wikipedia.org_all-access_all-agents
51439	Creed_(film)_fr.wikipedia.org_mobile-web_all-a...
15118	File:Olympic_rings_without_rims.svg_commons.wi...
18411	Покровская,_Татьяна_Николаевна_ru.wikipedia.or...
22915	User_hub/de_www.mediawiki.org_mobile-web_all-a...
56525	横尾初喜_ja.wikipedia.org_mobile-web_all-agents
66227	Mein_Kampf_de.wikipedia.org_desktop_all-agents
35625	Project_MKUltra_en.wikipedia.org_all-access_sp...
110644	Dormanstown_en.wikipedia.org_all-access_all-ag...

dtype: object

The page name contains data in the below format: SPECIFIC NAME _ LANGUAGE.wikipedia.org _ ACCESS TYPE _ ACCESS ORIGIN having information about page name, the domain, device type used to access the page, also the request origin(spider or browser agent).

```
In [199... # Extract 'name' and 'language' in a single step
df[['name', 'lang']] = df['Page'].str.extract(r'^(.*)_([a-z]{2})\.wikipedia\.
```

```
# Fill missing values if any pattern didn't match
df['name'] = df['name'].fillna('unknown')
df['lang'] = df['lang'].fillna('un')
```

```
In [200... import matplotlib.pyplot as plt

# Language mapping using modern dict literal
lang_name_mapping = {
    'zh': 'Chinese',
    'fr': 'French',
    'en': 'English',
    'un': 'Unknown',
    'ru': 'Russian',
    'de': 'German',
    'ja': 'Japanese',
    'es': 'Spanish'
}

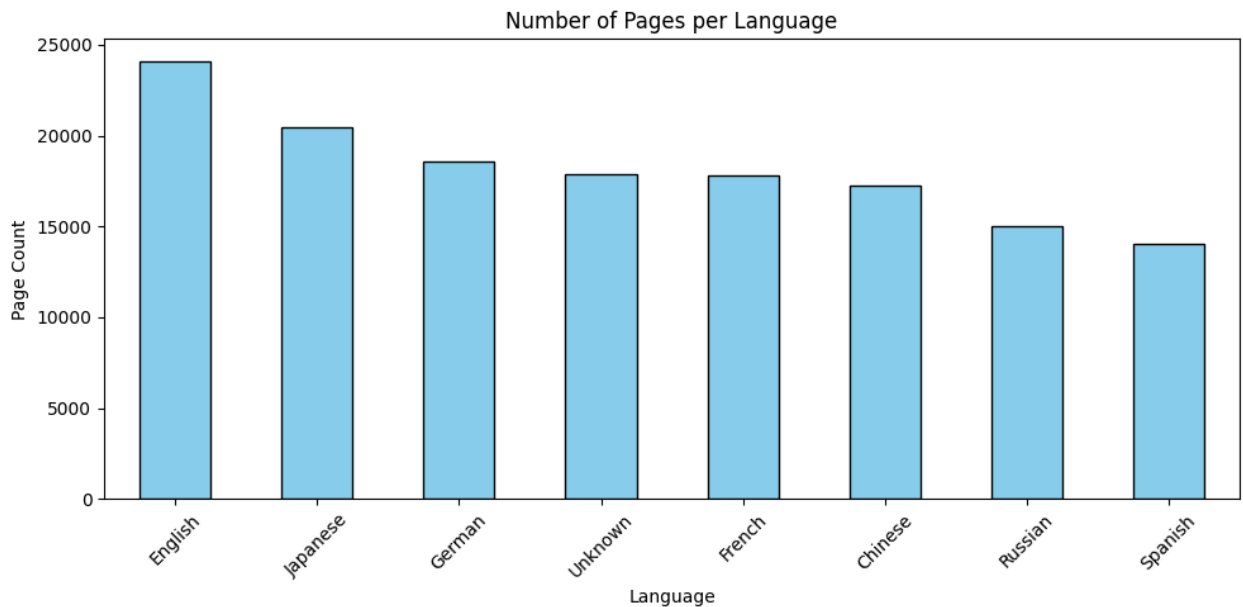
# Use .map with a fallback (optional but good practice)
df['language'] = df['lang'].map(lang_name_mapping).fillna('Other')
```

```

# Plotting with updated style
ax = df['language'].value_counts().plot(
    kind='bar',
    title='Number of Pages per Language',
    xlabel='Language',
    ylabel='Page Count',
    figsize=(10, 5),
    color='skyblue',
    edgecolor='black'
)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Display language distribution percentages
print("📊 Percentage of pages by language:")
language_percent = df['language'].value_counts(normalize=True).mul(100).round(
print(language_percent)

```



```

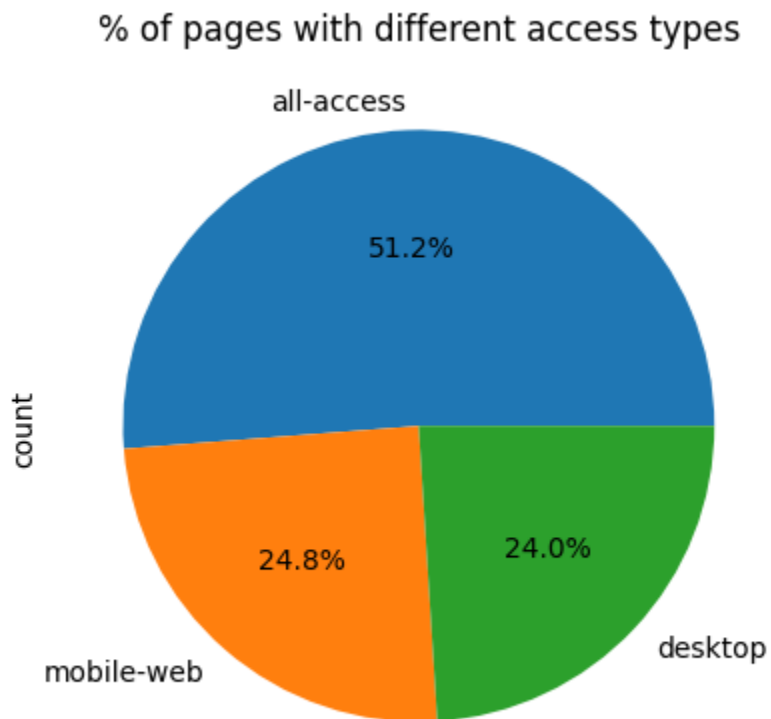
📊 Percentage of pages by language:
language
English      16.62
Japanese     14.08
German       12.79
Unknown      12.31
French       12.27
Chinese      11.88
Russian      10.36
Spanish       9.70
Name: proportion, dtype: float64

```

Insight

Maximum number of pages, 16.62%, are in English language

```
In [201... #Extracting access type
df['access_type'] = df['Page'].str.findall(r'all-access|mobile-web|desktop').a
df['access_type'].value_counts().plot(kind='pie', autopct='%1.1f%%', title='%
plt.show()
```

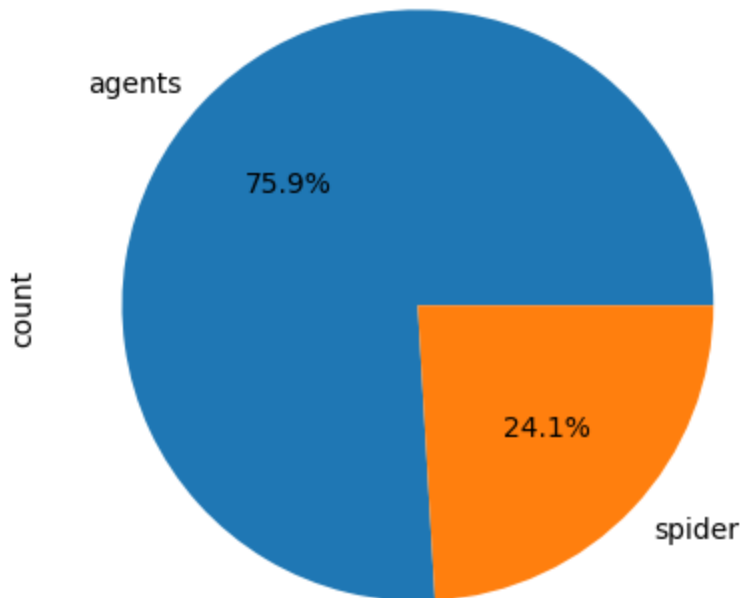


Insight

Maximum number of pages, 51.2%, have all-access access type

```
In [202... # Extracting access origin
df['access_origin'] = df['Page'].str.findall(r'spider|agents').apply(lambda x:
df['access_origin'].value_counts().plot(kind='pie', autopct='%1.1f%%', title='
plt.show()
```

% of pages with different access origin



Insight

Maximum number of pages, 75.9%, have agents access origin

5. Aggregate and Pivoting

In [203... `df.head()`

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04
0	2NE1_zh.wikipedia.org_all-access_spider	18.0	11.0	5.0	0.0
1	2PM_zh.wikipedia.org_all-access_spider	11.0	14.0	15.0	0.0
2	3C_zh.wikipedia.org_all-access_spider	1.0	0.0	1.0	0.0
3	4minute_zh.wikipedia.org_all-access_spider	35.0	13.0	10.0	0.0
4	52_Hz_I_Love_You_zh.wikipedia.org_all-access_spider	0.0	0.0	0.0	0.0

5 rows × 556 columns

In [204... `# Step 1: Identify view count columns (they should be date strings)`
`view_cols = df.columns.difference(['Page', 'name', 'lang', 'language', 'access'])`

```

# Step 2: Group by language and calculate daily average views
df_agg = (
    df.groupby('language')[view_cols]
      .mean()
      .T
      .reset_index()
      .rename(columns={'index': 'date'})
)

# Step 3: Convert 'date' column to datetime and set it as index
df_agg['date'] = pd.to_datetime(df_agg['date'], errors='coerce')
df_agg = df_agg.set_index('date')

# Preview result
df_agg.head()

```

```

Out[204...]

```

	language	Chinese	English	French	German	Japanese	Rus
	date						
	2015-07-01	240.582042	3513.862203	475.150994	714.968405	580.647056	629.99
	2015-07-02	240.941958	3502.511407	478.202000	705.229741	666.672801	640.90
	2015-07-03	239.344071	3325.357889	459.837659	676.877231	602.289805	594.02
	2015-07-04	241.653491	3462.054256	491.508932	621.145145	756.509177	558.72
	2015-07-05	257.779674	3575.520035	482.557746	722.076185	725.720914	595.02

5.1. Time series plots for all languages

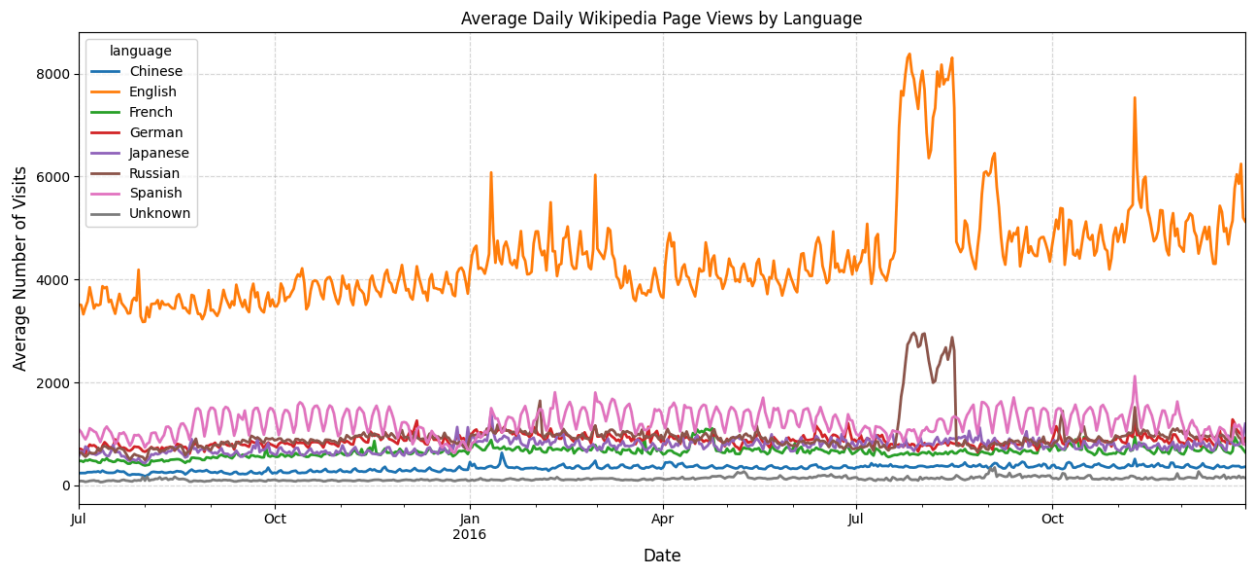
```

In [205...]
# Plot the aggregated view trends by language over time
ax = df_agg.plot(
    figsize=(13, 6),
    linewidth=2,
    title='Average Daily Wikipedia Page Views by Language',
)

# Labeling the axes
ax.set_xlabel('Date', fontsize=12)
ax.set_ylabel('Average Number of Visits', fontsize=12)

# Optional: Improve layout and grid
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```



Insights

English Wikipedia pages have the highest average daily visits overall.

Significant spikes in English page views occur around mid-2016, likely due to events or campaigns.

Spanish and Russian pages show moderate and fairly consistent visit levels.

Russian pages also show noticeable spikes around mid-2016.

Chinese, French, German, Japanese, and Unknown languages have steady but lower traffic.

Most languages exhibit relatively stable trends, except for some event-driven spikes in English and Russian pages.

6. Stationarity, Detrending, ACF and PACF

6.1. Stationarity test

Using Augmented Dickey-Fuller test to check for stationarity

H0: The series is not stationary

H1: The series is stationary

```
In [206... def adfuller_test(time_series, signif=0.05):
    result = sm.tsa.stattools.adfuller(time_series)
    p_value = result[1]
    if p_value < signif:
        print(f'Time series is stationary (p-value = {p_value:.4f})')
```

```

else:
    print(f'Time series is NOT stationary (p-value = {p_value:.4f})')

for lang in df_agg.columns:
    print(f"Testing stationarity for: {lang}")
    adfuller_test(df_agg[lang])
    print()

```

```

Testing stationarity for: Chinese
Time series is NOT stationary (p-value = 0.4474)

```

```

Testing stationarity for: English
Time series is NOT stationary (p-value = 0.1895)

```

```

Testing stationarity for: French
Time series is NOT stationary (p-value = 0.0515)

```

```

Testing stationarity for: German
Time series is NOT stationary (p-value = 0.1410)

```

```

Testing stationarity for: Japanese
Time series is NOT stationary (p-value = 0.1026)

```

```

Testing stationarity for: Russian
Time series is stationary (p-value = 0.0019)

```

```

Testing stationarity for: Spanish
Time series is stationary (p-value = 0.0336)

```

```

Testing stationarity for: Unknown
Time series is stationary (p-value = 0.0163)

```

Russian, Spanish, and Unknown languages show stationary behavior, with stable average visits over the period.

Most languages' page views (Chinese, English, French, German, Japanese) are not stationary, meaning their average visits change over time.

The non-stationarity in major languages suggests trends or seasonality in page views, which is important to consider for time series modeling.

Stationary series (Russian, Spanish, Unknown) might be easier to model without further transformations.

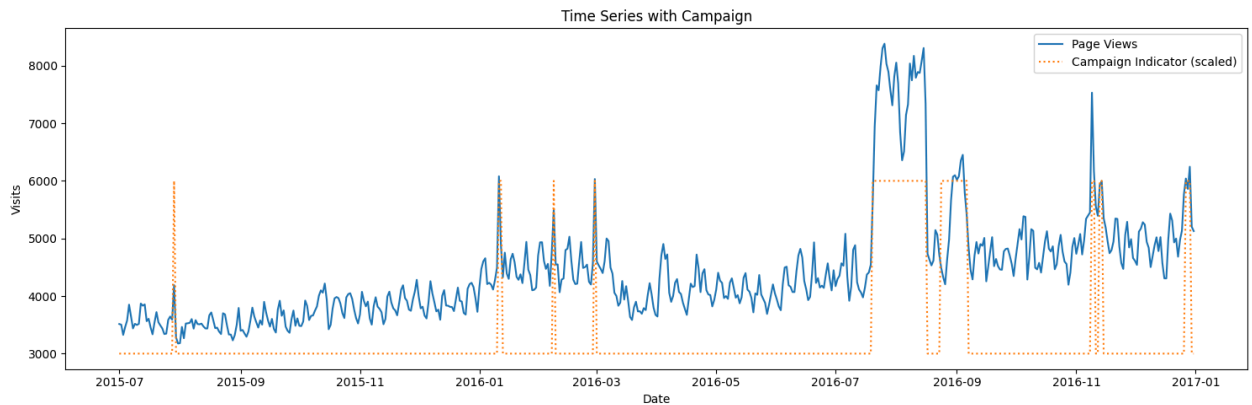
From now on, we will work only on the English language page visit time series

```
In [207... ts_english = df_agg['English']
```

```
In [208... # Let us look at the English time series along with its exog flag
def plot_ts_with_campaign(ts, exog, scale=3000, title='Time Series with Campai
```

```
plt.figure(figsize=(15, 5))
plt.plot(ts.index, ts, label='Page Views')
plt.plot(ts.index, (exog + 1) * scale, ':', label='Campaign Indicator (scaled)')
plt.title(title)
plt.xlabel('Date')
plt.ylabel('Visits')
plt.legend()
plt.tight_layout()
plt.show()

plot_ts_with_campaign(ts_english, exog_en)
```



Insights

The time series shows a steady trend in Wikipedia page views over time.

Noticeable spikes align closely with the campaign indicator (orange dotted line).

These spikes suggest campaigns significantly boost page visits.

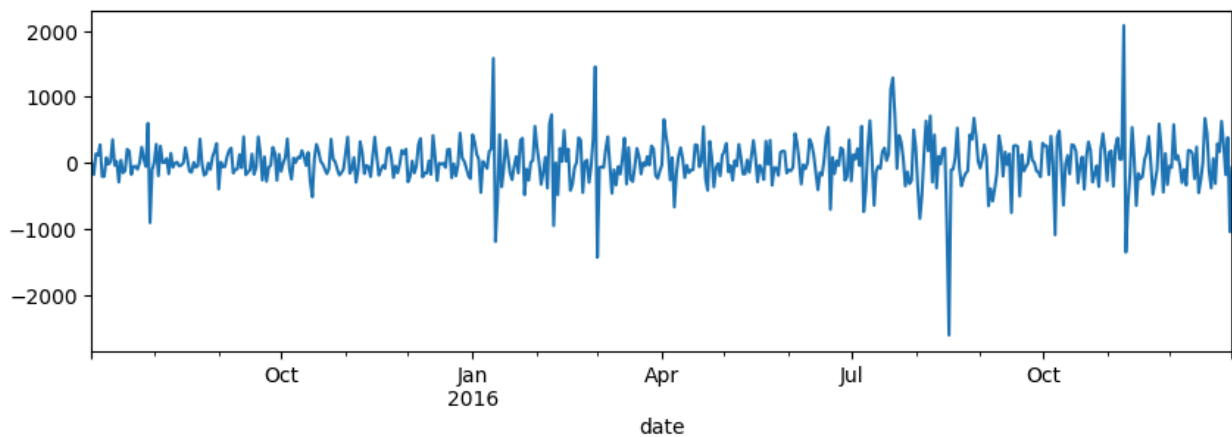
Between campaigns, page views remain relatively stable.

The plot clearly highlights the strong impact of special events on traffic.

6.2. De-trending and De-seasoning

As the trend is linear, differencing with the previous value should de-trend the time series

```
In [209... ts_english.diff(1).dropna().plot(figsize=(10,3))
plt.show()
```

Augmented Dickey-Fuller (ADF) test

The Augmented Dickey-Fuller test is a common statistical test used to check stationarity of a time series.

Stationarity means the statistical properties (like mean and variance) of the series do not change over time.

Stationary time series are crucial for many time series models (like ARIMA), which assume stationarity for valid predictions.

Some Wikipedia pages show trends or seasonal spikes in daily views over time.

The ADF test helps identify if these page view series are stationary or not.

For non-stationary pages, differencing can make the data stationary, which is essential for accurate forecasting.

Applying ADF on key languages or pages with clear trends guides how to prepare the data before modeling.

This ensures models like ARIMA and SARIMAX perform better and give reliable predictions for ad placements.

```
In [210... adfuller_test(ts_english.diff(1).dropna())
```

Time series is stationary (p-value = 0.0000)

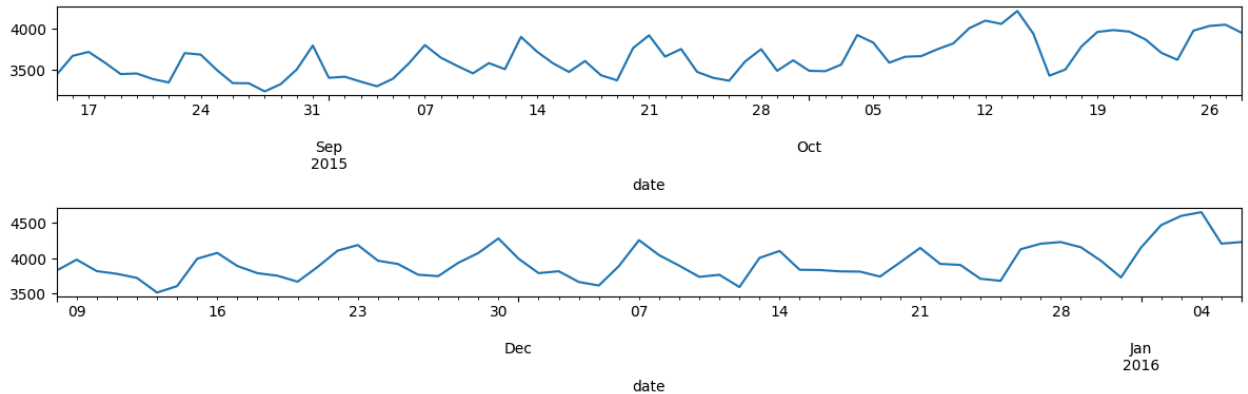
The time series became stationary by just doing first-order differencing, hence $d=1$

Let's now look at the seasonality

```
In [211... def plot_time_slices(ts, slices, figsize=(12, 2)):
    fig, axes = plt.subplots(len(slices), 1, figsize=(figsize[0], figsize[1]*l
    if len(slices) == 1:
        axes = [axes]
```

```
for ax, (start, end) in zip(axes, slices):
    ts[start:end].plot(ax=ax)
plt.tight_layout()
plt.show()
```

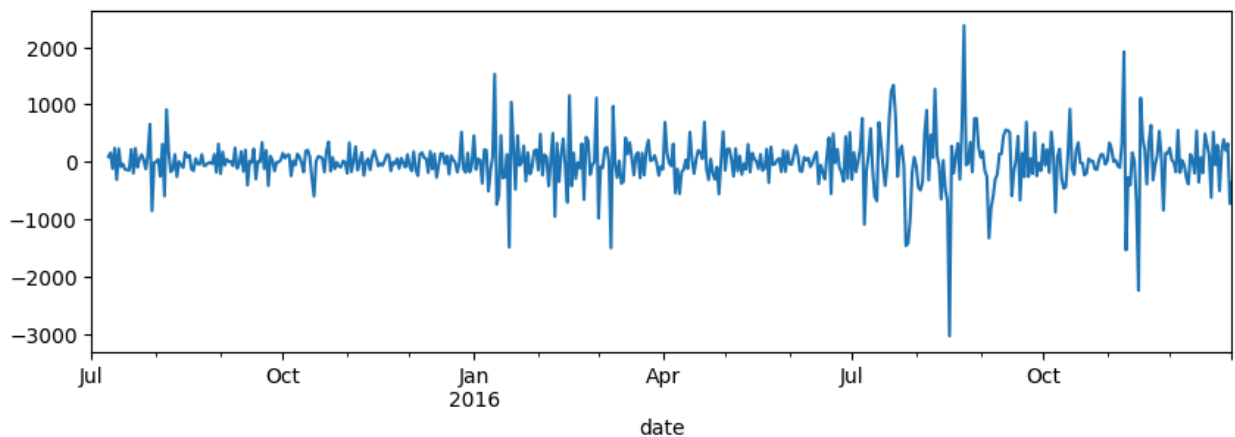
```
plot_time_slices(ts_english, [(45, 120), (130, 190)])
```



Observing the above two plots, we can conclude that there is a seasonality of 7 days. So $s=7$

The peaks and troughs repeat every 7 days

```
In [212... ts_english.diff(1).diff(7).plot(figsize=(10,3))
plt.show()
```



```
In [213... adfuller_test(ts_english.diff(1).diff(7).dropna())
```

Time series is stationary (p-value = 0.0000)

After removing the trend(and if required, seasonality) manually, the Augmented Dickey-Fuller test says that the time series is stationary

6.3. Auto de-composition

We had done manual decomposition above but there is a statsmodel library to

decompose time series

```
In [214... # Decompose the time series
decom = seasonal_decompose(ts_english)
components = {
    'Actual': ts_english,
    'Trend': decom.trend,
    'Seasonal': decom.seasonal,
    'Residual': decom.resid
}

# Plot all components
plt.figure(figsize=(15, 8))
for i, (title, series) in enumerate(components.items(), start=1):
    plt.subplot(4, 1, i)
    plt.plot(series, label=title.lower())
    plt.legend(loc='best')
    plt.title(title)

plt.tight_layout()
plt.show()
```



6.4. ACF and PACF Plots

The Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots are essential tools for identifying the order of ARIMA models.

ACF (Autocorrelation Function): Measures the correlation of a time series with its own lagged values. It helps determine the order of MA (Moving Average) components. Significant spikes at specific lags suggest potential MA terms.

PACF (Partial Autocorrelation Function): Measures the correlation between a time series and its lagged values after removing the influence of intermediate lags. It helps determine the order of AR (Auto-Regressive) components. Significant spikes indicate potential AR terms.

⚠ Note: Ensure the input series is stationary before using ACF or PACF.

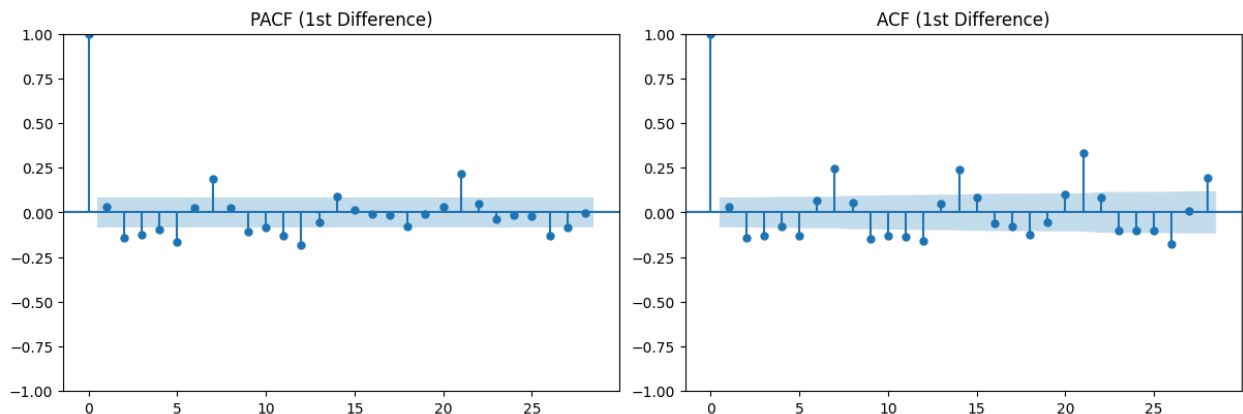
```
In [215... # Preprocess: Difference the series once and drop NaNs
ts_diff = ts_english.diff().dropna()

# Create side-by-side plots
fig, axs = plt.subplots(1, 2, figsize=(12, 4), constrained_layout=True)

plot_pacf(x=ts_diff, ax=axs[0], method='ywmm')
axs[0].set_title('PACF (1st Difference)')

plot_acf(x=ts_diff, ax=axs[1])
axs[1].set_title('ACF (1st Difference)')

plt.show()
```



From the PACF plot, we can see that there are 3 significant lags, at 5, 7 and 21. So $P=1,2$ or 3

From the ACF plot, we can see that there are 3 significant lags, at 7, 14 and 21. So $Q=1,2$ or 3

From the PACF plot, the cut-off is right from lag 0 and same for ACF plot. hence, p and $q = 0$ or 1

7. Model building and Evaluation

```
In [216... def performance(
    actual, predicted, show=True, round_digits=3
):
    """
    Evaluate forecast performance with MAE, RMSE, and MAPE.
```

```

"""
metrics = {
    'MAE': round(mae(actual, predicted), round_digits),
    'RMSE': round(np.sqrt(mse(actual, predicted)), round_digits),
    'MAPE': round(mape(actual, predicted), round_digits)
}

if show:
    for k, v in metrics.items():
        print(f"{k}: {v}")

return metrics

# === Forecast with ARIMA and evaluate ===
n_forecast = 60
ts = ts_english.copy()

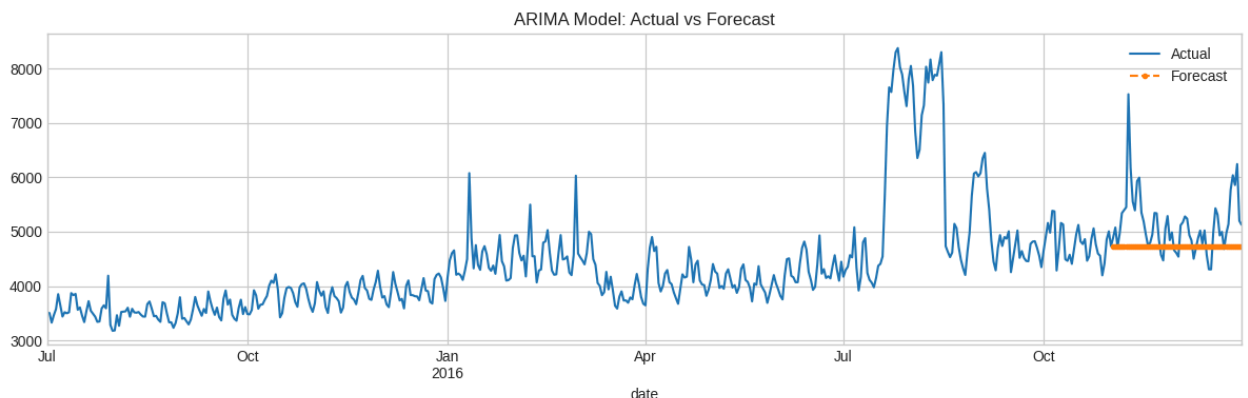
# Fit ARIMA model
model = ARIMA(ts[:-n_forecast], order=(0, 1, 0)).fit()

# Forecast
forecast = model.forecast(steps=n_forecast, alpha=0.05)

# Plot actual vs forecast
with plt.style.context("seaborn-v0_8-whitegrid"):
    plt.figure(figsize=(12, 4))
    ts.plot(label='Actual')
    forecast.plot(label='Forecast', linestyle='--', marker='.')
    plt.title('ARIMA Model: Actual vs Forecast')
    plt.legend(loc='upper right')
    plt.tight_layout()
    plt.show()

# Evaluate performance
_ = performance(ts.values[-n_forecast:], forecast.values)

```



MAE: 477.636
 RMSE: 672.778
 MAPE: 0.086

Insight

The model is not doing a good job, even for different combinations of p and q

7.2. SARIMAX model

```
In [217... from statsmodels.tsa.statespace.sarimax import SARIMAX
import matplotlib.pyplot as plt
import numpy as np

def split_data(ts, exog, n_forecast):
    """
    Split time series and exogenous data into training and test sets.
    """
    ts_train, ts_test = ts[:-n_forecast], ts[-n_forecast:]
    exog_train, exog_test = exog[:-n_forecast], exog[-n_forecast:]
    return ts_train, ts_test, exog_train, exog_test

def train_sarimax(ts_train, exog_train, order, seasonal_order):
    """
    Train SARIMAX model with given parameters and data.
    """
    model = SARIMAX(
        ts_train,
        exog=exog_train,
        order=order,
        seasonal_order=seasonal_order,
        initialization='approximate_diffuse'
    )
    model_fit = model.fit()
    return model_fit

def forecast_sarimax(model_fit, exog_forecast, n_forecast, dynamic=True):
    """
    Forecast using the SARIMAX model with future exogenous variables.
    """
    forecast = model_fit.forecast(steps=n_forecast, exog=exog_forecast, dynamic=dynamic)
    return forecast

def plot_forecast(ts, forecast, n_tail=120, order=None, seasonal_order=None):
    """
    Plot actual vs forecasted time series.
    """
    p, d, q = order
    P, D, Q, s = seasonal_order
    title = f'SARIMAX ({p},{d},{q}) x ({P},{D},{Q},{s}) Seasonal Model\nActual vs Forecast'

    with plt.style.context("seaborn-v0_8-whitegrid"):
        plt.figure(figsize=(14, 6))
        ts[-n_tail:].plot(label='Actual')
        forecast[-n_tail:].plot(
            label='Forecast',
            color='red',
            linestyle='--',
            marker='o',
```

```

        markerfacecolor='green'
    )
    plt.title(title)
    plt.legend(loc='upper right')
    plt.tight_layout()
    plt.show()

def run_sarimax_pipeline(ts, exog, order, seasonal_order, n_forecast):
    """
    Full SARIMAX pipeline: split data, train model, forecast, evaluate, plot.
    """
    # Split the data
    ts_train, ts_test, exog_train, exog_test = split_data(ts, exog, n_forecast)

    # Train model
    model_fit = train_sarimax(ts_train, exog_train, order, seasonal_order)

    # Forecast
    forecast = forecast_sarimax(model_fit, exog_test, n_forecast)

    # Plot
    plot_forecast(ts, forecast, order=order, seasonal_order=seasonal_order)

    # Evaluate
    metrics = performance(ts_test.values, forecast.values)
    return model_fit, forecast, metrics

```

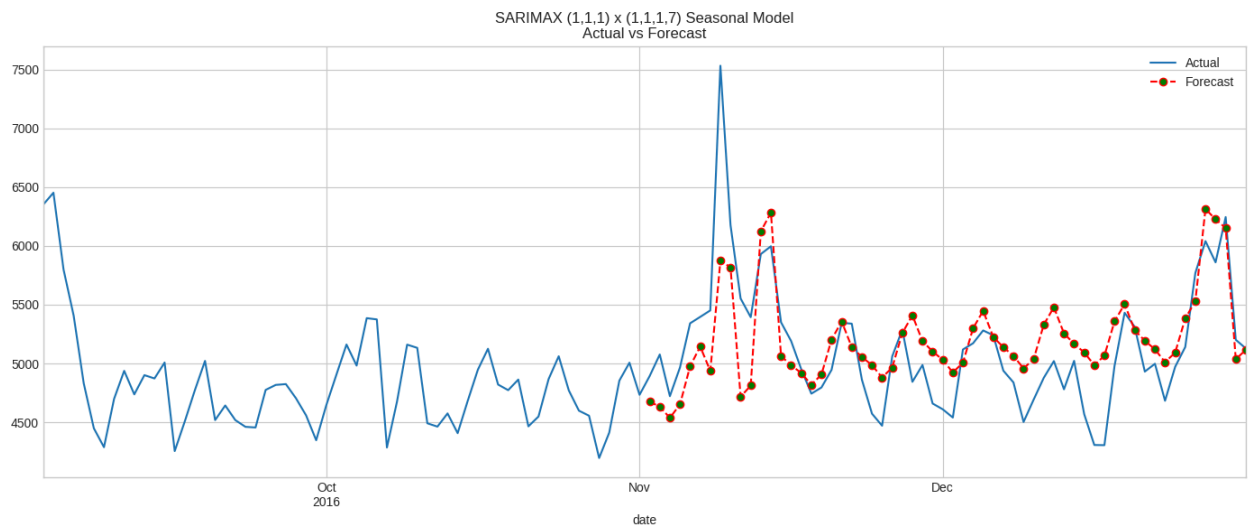
```

In [218...] TS = ts_english.copy() # Your time series pandas Series with datetime index
exog_array = exog_en['Exog'].to_numpy() # Convert exog to numpy array

order = (1, 1, 1)
seasonal_order = (1, 1, 1, 7)
n_forecast = 60

model_fit, forecast, metrics = run_sarimax_pipeline(
    ts=TS,
    exog=exog_array,
    order=order,
    seasonal_order=seasonal_order,
    n_forecast=n_forecast
)

```



MAE: 306.416
 RMSE: 399.015
 MAPE: 0.06

Insight

SARIMAX model is doing a significantly better job. We need to search for the right order values

Let tune our SARIMAX model orders to find the best parameters for our data by exhaustively fits models for every combination of specified parameters.

Problems with the current version:

Arguments are a bit confusing: forecast is listed but never used.

n_forecast is used inside but not passed as an argument.

The nested loops are very deeply nested (7-levels) — can get slow and hard to read.

No logging of exceptions (just continue), which makes debugging hard.

Uses deprecated method `loc[len(df)] = ...` (better to build a list of dicts then `pd.DataFrame` once).

```
In [219... def performance(true, pred):
    mae = mean_absolute_error(true, pred)
    rmse = np.sqrt(mean_squared_error(true, pred))
    mape = np.mean(np.abs((true - pred) / true)) * 100
    return mae, rmse, mape

def SARIMAX_search(TS, n_forecast,
                   p_list, d_list, q_list,
                   P_list, D_list, Q_list,
```



```

        s_list,
        exog=None,
        verbose=False):
    results = []
    total = len(p_list)*len(d_list)*len(q_list)*len(P_list)*len(D_list)*len(Q_list)
    count = 0

    for p in p_list:
        for d in d_list:
            for q in q_list:
                for P in P_list:
                    for D in D_list:
                        for Q in Q_list:
                            for s in s_list:
                                try:
                                    model = SARIMAX(
                                        TS[:-n_forecast],
                                        order=(p,d,q),
                                        seasonal_order=(P,D,Q,s),
                                        exog=exog[:-n_forecast] if exog is not None else None,
                                        initialization='approximate_diffuse'
                                    )
                                    model_fit = model.fit(dispatch=False)

                                    forecast = model_fit.forecast(
                                        steps=n_forecast,
                                        exog=exog[-n_forecast:] if exog is not None else None,
                                        dynamic=True
                                    )
                                    mae, rmse, mape = performance(TS[-n_forecast:], forecast)
                                    results.append({
                                        'pdq': (p,d,q),
                                        'PDQs': (P,D,Q,s),
                                        'MAE': mae,
                                        'RMSE': rmse,
                                        'MAPE': mape
                                    })
                                    count += 1
                                    if verbose:
                                        print(f"Tested {count}/{total}: order {p,d,q} seasonal_order {P,D,Q,s}")
                                except Exception as e:
                                    if verbose:
                                        print(f"Failed {(p,d,q)} {(P,D,Q,s)}: {e}")
                                    continue

    return pd.DataFrame(results)

# Plotting function
def plot_SARIMAX_forecast(TS, n_forecast, order, seasonal_order, exog=None, title=None):
    model = SARIMAX(
        TS[:-n_forecast],
        order=order,
        seasonal_order=seasonal_order,

```

```

        exog=exog[:-n_forecast] if exog is not None else None,
        initialization='approximate_diffuse'
    )
    model_fit = model.fit(dispatch=False)
    forecast = model_fit.forecast(
        steps=n_forecast,
        exog=exog[-n_forecast:] if exog is not None else None,
        dynamic=True
    )
    plt.figure(figsize=(14,6))
    TS[-n_forecast*2:].plot(label='Actual')
    forecast.plot(label='Forecast', color='red', linestyle='--', marker='o', m
    plt.legend()
    if title:
        plt.title(title)
    plt.show()

```

```

In [220... exog = exog_en['Exog'].to_numpy()
results_df = SARIMAX_search(
    TS, n_forecast,
    p_list, d_list, q_list,
    P_list, D_list, Q_list,
    s_list,
    exog=exog,
    verbose=True
)

print(results_df.sort_values(['MAPE', 'RMSE']).head())

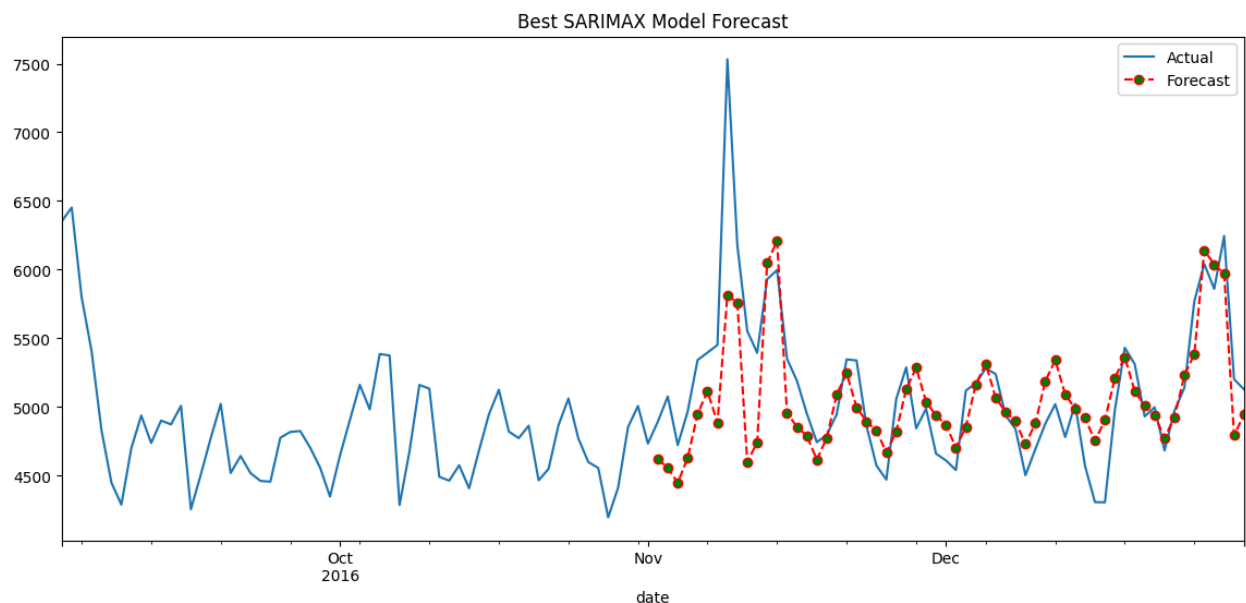
best = results_df.sort_values(['MAPE', 'RMSE']).iloc[0]

plot_SARIMAX_forecast(
    TS, n_forecast,
    order=best['pdq'],
    seasonal_order=best['PDQs'],
    exog=exog,
    title='Best SARIMAX Model Forecast'
)

```

Tested 1/16: order (0, 1, 0) seasonal (2, 1, 2, 7) MAPE=6.428
 Tested 2/16: order (0, 1, 0) seasonal (2, 1, 3, 7) MAPE=6.260
 Tested 3/16: order (0, 1, 0) seasonal (3, 1, 2, 7) MAPE=6.242
 Tested 4/16: order (0, 1, 0) seasonal (3, 1, 3, 7) MAPE=6.061
 Tested 5/16: order (0, 1, 1) seasonal (2, 1, 2, 7) MAPE=6.395
 Tested 6/16: order (0, 1, 1) seasonal (2, 1, 3, 7) MAPE=5.707
 Tested 7/16: order (0, 1, 1) seasonal (3, 1, 2, 7) MAPE=6.199
 Tested 8/16: order (0, 1, 1) seasonal (3, 1, 3, 7) MAPE=6.060
 Tested 9/16: order (1, 1, 0) seasonal (2, 1, 2, 7) MAPE=6.395
 Tested 10/16: order (1, 1, 0) seasonal (2, 1, 3, 7) MAPE=5.611
 Tested 11/16: order (1, 1, 0) seasonal (3, 1, 2, 7) MAPE=6.188
 Tested 12/16: order (1, 1, 0) seasonal (3, 1, 3, 7) MAPE=5.628
 Tested 13/16: order (1, 1, 1) seasonal (2, 1, 2, 7) MAPE=5.692
 Tested 14/16: order (1, 1, 1) seasonal (2, 1, 3, 7) MAPE=5.114
 Tested 15/16: order (1, 1, 1) seasonal (3, 1, 2, 7) MAPE=5.920
 Tested 16/16: order (1, 1, 1) seasonal (3, 1, 3, 7) MAPE=5.718

	pdq	PDQs	MAE	RMSE	MAPE
13	(1, 1, 1)	(2, 1, 3, 7)	269.231674	373.568019	5.114020
9	(1, 1, 0)	(2, 1, 3, 7)	300.249403	412.034267	5.611179
11	(1, 1, 0)	(3, 1, 3, 7)	301.122997	411.758540	5.627880
12	(1, 1, 1)	(2, 1, 2, 7)	293.031998	381.953154	5.691751
5	(0, 1, 1)	(2, 1, 3, 7)	305.642917	416.966059	5.706916



Insight

There is good improvement in the SARIMAX model after tuning the parameters

7.4. Facebook Prophet

```
In [221... !pip install pystan==2.19.1.1
!pip install prophet
```

```

Collecting pystan==2.19.1.1
  Using cached pystan-2.19.1.1.tar.gz (16.2 MB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: Cython!=0.25.1,>=0.22 in /usr/local/lib/python3.12/dist-packages (from pystan==2.19.1.1) (3.0.12)
Requirement already satisfied: numpy>=1.7 in /usr/local/lib/python3.12/dist-packages (from pystan==2.19.1.1) (2.0.2)
Building wheels for collected packages: pystan
  error: subprocess-exited-with-error

  × python setup.py bdist_wheel did not run successfully.
  | exit code: 1
  |_ See above for output.

```

note: This error originates from a subprocess, and is likely not a problem with pip.

```

Building wheel for pystan (setup.py) ... error
ERROR: Failed building wheel for pystan
Running setup.py clean for pystan
Failed to build pystan
ERROR: ERROR: Failed to build installable wheels for some pyproject.toml based projects (pystan)
Requirement already satisfied: prophet in /usr/local/lib/python3.12/dist-packages (1.1.7)
Requirement already satisfied: cmdstanpy>=1.0.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (1.2.5)
Requirement already satisfied: numpy>=1.15.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (2.0.2)
Requirement already satisfied: matplotlib>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from prophet) (3.10.0)
Requirement already satisfied: pandas>=1.0.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (2.2.2)
Requirement already satisfied: holidays<1,>=0.25 in /usr/local/lib/python3.12/dist-packages (from prophet) (0.81)
Requirement already satisfied: tqdm>=4.36.1 in /usr/local/lib/python3.12/dist-packages (from prophet) (4.67.1)
Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages (from prophet) (6.5.2)
Requirement already satisfied: stanio<2.0.0,>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from cmdstanpy>=1.0.4->prophet) (0.5.1)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from holidays<1,>=0.25->prophet) (2.9.0.post0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (4.60.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (11.3.0)

```

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (3.2.4)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.4->prophet) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.4->prophet) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->holidays<1,>=0.25->prophet) (1.17.0)

```
In [222... def prepare_ts_dataframe(ts_df, language_column, exog_series=None):
    """
    Prepare a DataFrame for Prophet with columns 'ds', 'y', and optional 'exog'

    Parameters:
    - ts_df: original time series DataFrame with date as index or a column
    - language_column: string, name of the column with the language data (e.g.
    - exog_series: optional pd.Series of exogenous variables, aligned with ts_

    Returns:
    - TS: DataFrame with 'ds', 'y', and optionally 'exog'
    """
    TS = ts_df.copy(deep=True).reset_index()

    # Identify the date/time column (first column after reset_index)
    date_col = TS.columns[0]

    # Select only date column and the language column
    TS = TS[[date_col, language_column]]

    # Rename columns for Prophet
    TS.columns = ['ds', 'y']

    # Convert date column to datetime
    TS['ds'] = pd.to_datetime(TS['ds'])

    # Add exogenous variable if provided
    if exog_series is not None:
        TS['exog'] = exog_series.values

    return TS

# Example usage:
TS = prepare_ts_dataframe(ts_english, language_column='English', exog_series=e
TS.tail()
```

Out[222...

	ds	y	exog
545	2016-12-27	6040.680728	1
546	2016-12-28	5860.227559	1
547	2016-12-29	6245.127510	1
548	2016-12-30	5201.783018	0
549	2016-12-31	5127.916418	0

In [223...

```
def prepare_prophet_model(daily_seasonality=False, weekly_seasonality=True, yearly_seasonality=True, interval_width=0.05):
    """
    Initialize a Prophet model with custom seasonality and interval width.
    """
    model = Prophet(
        interval_width=interval_width,
        daily_seasonality=daily_seasonality,
        weekly_seasonality=weekly_seasonality,
        yearly_seasonality=yearly_seasonality
    )
    return model

def add_regressors_to_model(model, regressors):
    """
    Add extra regressors to the Prophet model.
    regressors: list of string column names in your dataframe to add as regressors
    """
    for reg in regressors:
        model.add_regressor(reg)
    return model

def fit_prophet_model(model, df):
    """
    Fit the Prophet model on the dataframe df.
    df must have columns 'ds' (datetime), 'y' (target), plus any regressors.
    """
    model.fit(df)
    return model

def make_forecast(model, df, periods=0, regressors_df=None):
    future = model.make_future_dataframe(periods=periods)
    if regressors_df is not None:
        # Extend regressors_df to match future length
        current_len = len(regressors_df)
        future_len = len(future)

        if current_len < future_len:
            # Fill missing values (can choose method: zeros, forward fill, mean fill)
            fill_len = future_len - current_len
            fill_values = pd.DataFrame(
                {col: [regressors_df[col].iloc[-1]] * fill_len for col in regressors_df.columns}
            )
```

```

    )
    regressors_df = pd.concat([regressors_df, fill_values], ignore_index=1)

    for col in regressors_df.columns:
        future[col] = regressors_df[col].values
    forecast = model.predict(future)
    return forecast

def merge_forecast_with_actual(df, forecast):
    """
    Merge forecasted values back into original DataFrame for comparison and plotting.
    Returns the original DataFrame with added columns 'yhat', 'yhat_upper', 'yhat_lower'.
    """
    df = df.copy()
    df['yhat'] = forecast['yhat'].values[:len(df)]
    df['yhat_upper'] = forecast['yhat_upper'].values[:len(df)]
    df['yhat_lower'] = forecast['yhat_lower'].values[:len(df)]
    return df

def plot_actual_vs_forecast(df, title='Actual vs Predicted Visits'):
    """
    Plot the actual vs predicted values along with confidence intervals.
    Expects columns 'ds', 'y', 'yhat', 'yhat_upper', 'yhat_lower' in df.
    """
    plt.figure(figsize=(15, 5))
    plt.plot(df['ds'], df['y'], label='Actual Visits', color='blue')
    plt.plot(df['ds'], df['yhat'], label='Predicted Visits', color='red', alpha=0.5)
    plt.fill_between(df['ds'], df['yhat_lower'], df['yhat_upper'], color='pink', alpha=0.5)
    plt.xlabel('Date')
    plt.ylabel('Number of Visits')
    plt.title(title)
    plt.legend()
    plt.show()

def prophet_pipeline(TS, regressors=None, n_forecast=0, performance_func=None):
    """
    Complete pipeline to train Prophet, forecast, evaluate, and plot.
    TS: pd.DataFrame with 'ds', 'y', and optional regressor columns
    regressors: list of regressor column names to include
    n_forecast: number of future periods to forecast (0 means only fit on TS)
    performance_func: function(actual, predicted, print_metrics=True) -> tuple of metrics

    Returns: model_fit, forecast_df, df_with_predictions, metrics (if performance_func is not None)
    """
    # Prepare model
    model = prepare_prophet_model()

    # Add regressors if any
    if regressors:
        model = add_regressors_to_model(model, regressors)

    # Fit model

```

```

model = fit_prophet_model(model, TS)

# Prepare regressors for future if forecasting beyond TS
regressors_df = TS[regressors] if regressors else None

# Forecast
forecast = make_forecast(model, TS, periods=n_forecast, regressors_df=regr

# Merge predictions
df_with_preds = merge_forecast_with_actual(TS, forecast)

# Evaluate if function provided
metrics = None
if performance_func:
    metrics = performance_func(df_with_preds['y'], df_with_preds['yhat'])

# Plot results
plot_actual_vs_forecast(df_with_preds)

return model, forecast, df_with_preds, metrics

```

In [224... *# Example setup assuming your TS DataFrame has 'ds', 'y', and 'exog' columns*

```

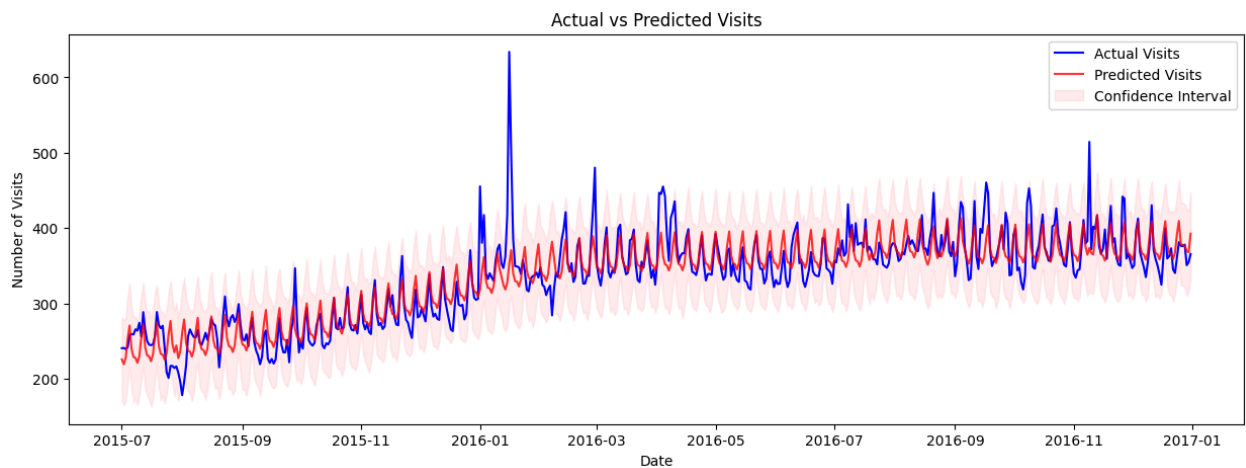
regressors = ['exog']
n_forecast = 60
TS = prepare_any_ts_dataframe(df_agg, language_column='Chinese', exog_series=e
model, forecast, df_with_preds, metrics = prophet_pipeline(
    TS,
    regressors=regressors,
    n_forecast=n_forecast,
    performance_func=performance # your evaluation function
)

```

```

DEBUG:cmdstanpy:input tempfile: /tmp/tmp114mqxgp/4k8a0t92.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmp114mqxgp/lydj4eyd.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophe
t/stan_model/prophet_model.bin', 'random', 'seed=35955', 'data', 'file=/tmp/tmp
114mqxgp/4k8a0t92.json', 'init=/tmp/tmp114mqxgp/lydj4eyd.json', 'output', 'fil
e=/tmp/tmp114mqxgp/prophet_modelqllvmtg8/prophet_model-20250925132940.csv', 'me
thod=optimize', 'algorithm=lbfgs', 'iter=10000']
13:29:40 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
13:29:40 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing

```

Insight

Phropet is doing an incredible job capturing the trend and unusual peaks. It is also capturing the seasonality very well

Let's explore and compare with few more languages

```
In [225... def generate_exog_from_ts(ts_series, window=7):
    """
    Generate an exogenous variable (e.g., rolling mean) from the time series i

    Parameters:
    - ts_series: pd.Series of the language time series
    - window: int, the window size for the rolling mean

    Returns:
    - pd.DataFrame with a single 'Exog' column
    """
    exog = ts_series.rolling(window=window, min_periods=1).mean()
    return pd.DataFrame({'Exog': exog})
```

```
In [226... def prepare_any_ts_dataframe(df_agg, language_column, exog_series=None):
    """
    Prepare a Prophet-compatible DataFrame (with 'ds', 'y', and optional 'exog'
    for a given language from the aggregated dataframe.

    Parameters:
    - df_agg: pd.DataFrame, original aggregated time series (columns: language
    - language_column: str, e.g., 'English', 'Chinese', etc.
    - exog_series: pd.Series or None, optional exogenous variable matching the

    Returns:
    - pd.DataFrame with columns: ['ds', 'y'] and optionally ['exog']
    """
    # Extract the time series for the language and reset index
    ts = df_agg[language_column].copy(deep=True).reset_index()
    ts = ts[[ts.columns[0], language_column]]
```

```

ts.columns = ['ds', 'y']
ts['ds'] = pd.to_datetime(ts['ds'])

# Attach exogenous variable if provided
if exog_series is not None:
    exog_series = exog_series.reset_index(drop=True)
    if len(exog_series) != len(ts):
        raise ValueError(f"Length mismatch: exog_series ({len(exog_series)})
    ts['exog'] = exog_series.values

return ts

```

Chinese

```

In [227...] exog_ch = generate_exog_from_ts(df_agg['Chinese'])
TS = prepare_any_ts_dataframe(df_agg, language_column='Chinese', exog_series=exog_ch)

```

```

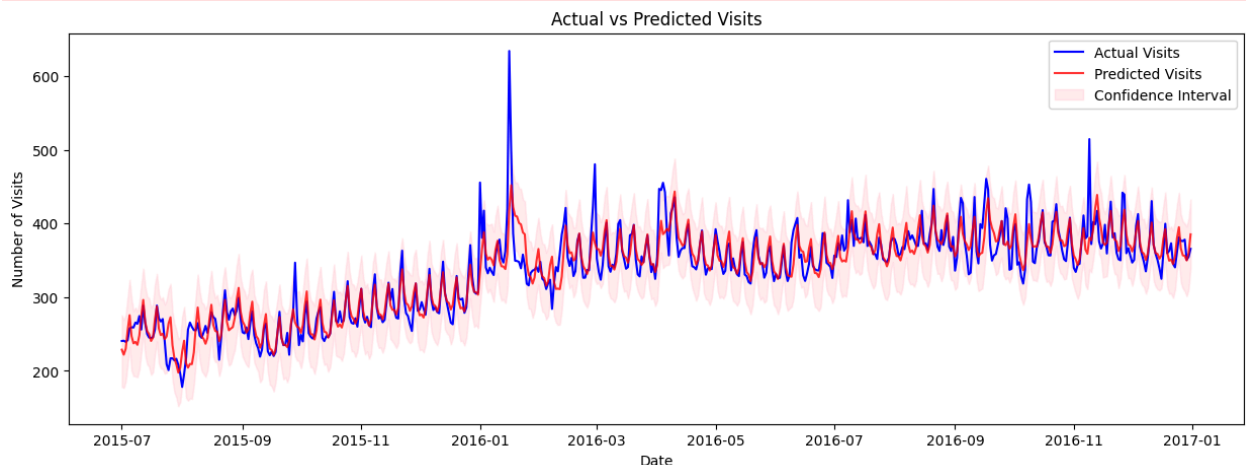
In [228...] model, forecast, df_with_preds, metrics = prophet_pipeline(
    TS,
    regressors=regressors,
    n_forecast=n_forecast,
    performance_func=performance
)

```

```

DEBUG:cmdstanpy:input tempfile: /tmp/tmpl14mqxgp/falgr0qt.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpl14mqxgp/s8jhjzqd.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_model.bin', 'random', 'seed=53203', 'data', 'file=/tmp/tmpl14mqxgp/falgr0qt.json', 'init=/tmp/tmpl14mqxgp/s8jhjzqd.json', 'output', 'file=/tmp/tmpl14mqxgp/prophet_model2i8ozpbq/prophet_model-20250925132941.csv', 'method=optimize', 'algorithm=lbfgs', 'iter=10000']
13:29:41 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
13:29:41 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing

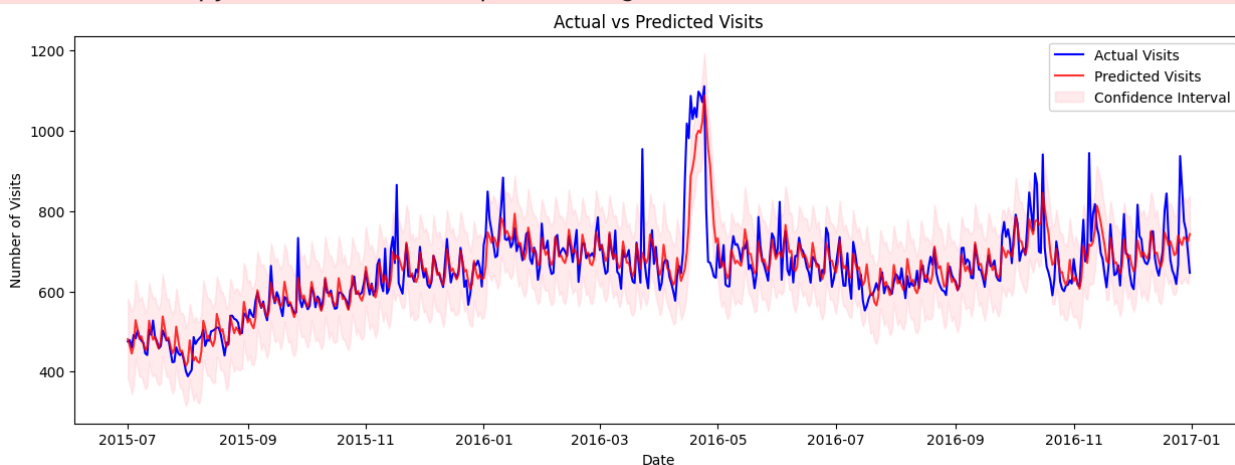
```



French

```
In [229... exog_fr = generate_exog_from_ts(df_agg['French'])
TS = prepare_any_ts_dataframe(df_agg, language_column='French', exog_series=ex
model, forecast, df_with_preds, metrics = prophet_pipeline(
    TS,
    regressors=regressors,
    n_forecast=n_forecast,
    performance_func=performance
)
```

```
DEBUG:cmdstanpy:input tempfile: /tmp/tmpl14mqxgp/43s4xllo.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpl14mqxgp/0pptnds6.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophe
t/stan_model/prophet_model.bin', 'random', 'seed=17911', 'data', 'file=/tmp/tmp
l14mqxgp/43s4xllo.json', 'init=/tmp/tmpl14mqxgp/0pptnds6.json', 'output', 'fil
e=/tmp/tmpl14mqxgp/prophet_modelalgubc9d/prophet_model-20250925132941.csv', 'me
thod=optimize', 'algorithm=lbfgs', 'iter=10000']
13:29:41 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
13:29:41 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
```



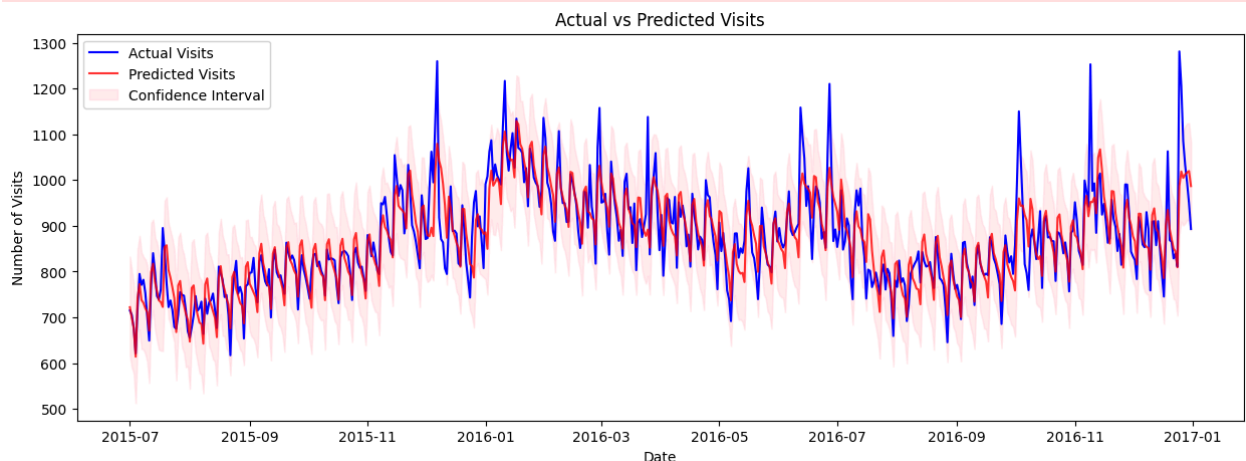
German

```
In [230... exog_german = generate_exog_from_ts(df_agg['German'])
TS = prepare_any_ts_dataframe(df_agg, language_column='German', exog_series=ex
model, forecast, df_with_preds, metrics = prophet_pipeline(
    TS,
    regressors=regressors,
    n_forecast=n_forecast,
    performance_func=performance
)
```

```

DEBUG:cmdstanpy:input tempfile: /tmp/tmp114mqxgp/iu9duqqd.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmp114mqxgp/kqawljmj.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_model.bin', 'random', 'seed=40167', 'data', 'file=/tmp/tmp114mqxgp/iu9duqqd.json', 'init=/tmp/tmp114mqxgp/kqawljmj.json', 'output', 'file=/tmp/tmp114mqxgp/prophet_modelgkyc5spe/prophet_model-20250925132942.csv', 'method=optimize', 'algorithm=lbfgs', 'iter=10000']
13:29:42 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
13:29:42 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing

```



Result Interpretations

 Languages in the Dataset The dataset contains data for 4 languages: English, German, French, and Chinese

 English Leads the Pack

English has the highest number of pages (16.62%).

This makes sense since English is the most widely used language online.

 Understanding Time Series with Decomposition

Time series data can be broken down into:

- Trend (long-term direction)

- Seasonality (repeating patterns like weekly or monthly)

- Noise (random variation)

This helps us better understand the behavior of the data over time.

Making the English Series Stationary

The English time series becomes stationary after applying differencing once.

This was confirmed using a statistical test called the ADF test (Augmented Dickey-Fuller).

Role of Exogenous Variables (External Events)

There's a special variable (called exogenous) that marks important events or campaigns.

We see spikes in visits whenever this variable is 1, showing that events clearly influence traffic.

What AdEase Can Do

AdEase can improve its forecasting by using models like Prophet that include exogenous variables.

This helps predict user behavior more accurately, especially during events or promotions.

Without Exogenous Variables, Accuracy Drops

For other languages (like Chinese or French), where no exogenous variables are given, the predictions are less accurate.

This shows that event data is important for good forecasting.

Difference Between Models (ARIMA Family)

ARIMA: Good for data with trends but no seasonality.

SARIMA: Adds support for seasonal patterns (e.g. weekly spikes).

SARIMAX: Adds support for external factors (exogenous variables), like campaigns or events. ➤ Best for data influenced by outside events.